O que é um Enum em java

o que é um Enum em java

Um **enum** (abreviação de **enumeração**) em Java é um tipo de dado especial que define um conjunto fixo de constantes nomeadas. [1, 2]

Ele serve para representar valores que não mudam durante a execução do programa, como os dias da semana, meses do ano, direções cardeais ou status de um pedido (ex: `PENDENTE`, `APROVADO`, `CANCELADO`). [1, 2, 3]

Sintaxe básica

Para criar um enum, utiliza-se a palavra-chave `enum` no lugar de `class`: [1]

java

```java
public enum DiaSemana {
    SEGUNDA,
    TERCA,
    QUARTA,
    QUINTA,
    SEXTA,
    SABADO,
    DOMINGO
}
```

Use o código com cuidado.

Por que usar enums?

- **Segurança de tipo (*Type Safety*):** Impede que uma variável receba valores inválidos que não estejam declarados na lista (diferente de usar apenas `String` ou `int`).
- **Legibilidade:** Deixa o código mais limpo, autoexplicativo e fácil de manter.
- **Uso com `switch`:** Enums funcionam muito bem em estruturas de decisão. [1, 2, 3, 4]

Principais características

- **Herança:** Todos os enums estendem implicitamente a classe `java.lang.Enum`. [1]
- **Métodos úteis herdados:**

- `values()`: retorna um array com todas as constantes do enum.
- `valueOf(String)`: converte uma string no valor correspondente do enum.
- `ordinal()`: retorna a posição (índice) da constante na declaração (começando em 0). [1, 2]

- **Atributos e Construtores:** Em Java, enums podem ter campos, métodos e construtores (que são sempre `private`) para associar dados extras a cada constante. [1, 2]

Você pode aprofundar o entendimento sobre a sintaxe e a criação dessas estruturas através do guia da DevMedia ou consultar conceitos adicionais na documentação da DataCamp. [1, 2]

Você deve usar um **enum** sempre que precisar representar um **conjunto fixo, predefinido e imutável de valores conhecidos em tempo de compilação**.

Se os dados mudam de forma dinâmica (como registros em um banco de dados), use classes e objetos normais. Se a lista é estrita e imutável, o enum é a escolha ideal.

Casos de uso mais comuns

- **Estados de um processo (Status):**
  - *Exemplos:* `PENDENTE`, `PROCESSANDO`, `APROVADO`, `CANCELADO`.
  - *Exemplos:* `RASCUNHO`, `PUBLICADO`, `ARQUIVADO`.
- **Dados da natureza ou geográficos:**
  - *Exemplos:* Dias da semana (`SEGUNDA` a `DOMINGO`), meses do ano, estações.
  - *Exemplos:* Direções cardeais (`NORTE`, `SUL`, `LESTE`, `OESTE`).
- **Configurações e Categorias fixas:**
  - *Exemplos:* Tamanhos de roupa (`P`, `M`, `G`, `GG`).
  - *Exemplos:* Tipos de usuário (`ADMINISTRADOR`, `CLIENTE`, `SUPORTE`).
  - *Exemplos:* Regimes fiscais (`SIMPLES_NACIONAL`, `LUCRO_PRESUMIDO`, `LUCRO_REAL`).

Sinais de que você deve trocar `String` ou `int` por um Enum

Se o seu código apresenta algum dos padrões abaixo, ele está pedindo a refatoração para um enum:

1. Evitar strings mágicas ou literais espalhadas

- **Ruim (Sem enum):**

  java

```
if (pedido.getStatus().equals("APROVADO")) { ... } // Risco de digitar "aprovado" ou "Aprovado" e quebrar o código
```

Use o código com cuidado.

- **Bom (Com enum):**

  java

  ```
  if (pedido.getStatus() == StatusPedido.APROVADO) { ... } // Seguro contra erros de digitação
  ```

  Use o código com cuidado.

2. Evitar o antipadrão de "Interfaces de Constantes" ou inteiros sem significado

- **Ruim (Sem enum):**

  java

  ```
  int SEGUNDA = 1;
  int TERCA = 2; // O que acontece se alguém passar o número 99 na função? O Java aceita.
  ```

  Use o código com cuidado.

- **Bom (Com enum):**

  java

  ```
  public void agendar(DiaSemana dia) { ... } // O compilador só aceita os dias válidos do seu enum.
  ```

  Use o código com cuidado.